

MF20006: Introduction to Computer Science

Lecture 4: Programming Languages

Hui Xu

xuh@fudan.edu.cn



Outline

1. Overview of Programming Languages
2. Python Basics
3. Agentic Coding with Python

1. Overview of Programming Languages

Concept of Programming Language

*“Applied mathematics is largely concerned with the design and analysis of **explicit procedures for calculating** the exact or approximate values of various functions.*

*Such explicit procedures are called **algorithms or programs**.*

*Because an **effective notation for the description** of programs exhibits considerable syntactic structure, it is called a **programming language**.”*

- Kenneth Iverson



From Mathematics to Program Languages

$$f(x) = \begin{cases} 1, & x = 1 \\ x * (f(x) - 1), & x > 1 \end{cases}$$

Math Formula

```
(defun fac (n)
  (if (= n 1)
      1
      (* n (fac (- n 1)))))
```

Lisp Code

(functional language)

```
def fac(n):
    if n == 1:
        return 1
    else:
        return n * fac(n - 1)
```

Python Code

(imperative language)

Popular Programming Languages: TIOBE Index

Sep 2026	Sep 2025	Change	Programming Language		Ratings	Change
1	1			Python	17.76%	-8.22%
2	3	^		C	10.28%	+1.63%
3	2	v		C++	8.67%	-0.13%
4	4			Java	7.54%	-0.81%
5	5			C#	4.22%	-2.16%
6	6			JavaScript	2.76%	-0.46%
7	7			Visual Basic	2.55%	-0.28%
8	11	^		SQL	2.16%	+0.29%
9	13	^^		R	1.69%	+0.27%
10	18	^^		Rust	1.34%	+0.33%
11	12	^		Fortran	1.24%	-0.25%
12	8	v		Go	1.10%	-1.22%
13	9	v		Delphi/Object Pascal	1.08%	-1.18%
14	15	^		PHP	1.04%	-0.21%
15	16	^		Scratch	0.99%	-0.19%
16	17	^		Assembly language	0.89%	-0.15%

Programming Languages We Generally Talked About

❑ High-level programming languages, like Python, C, Rust.

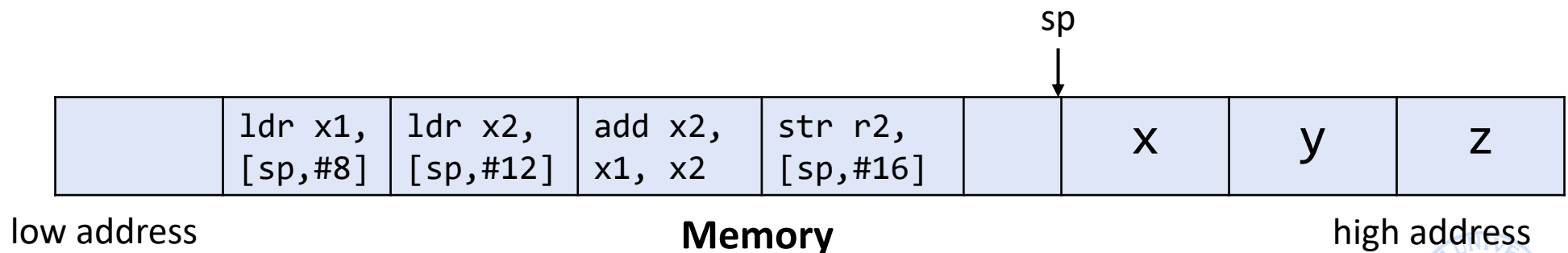
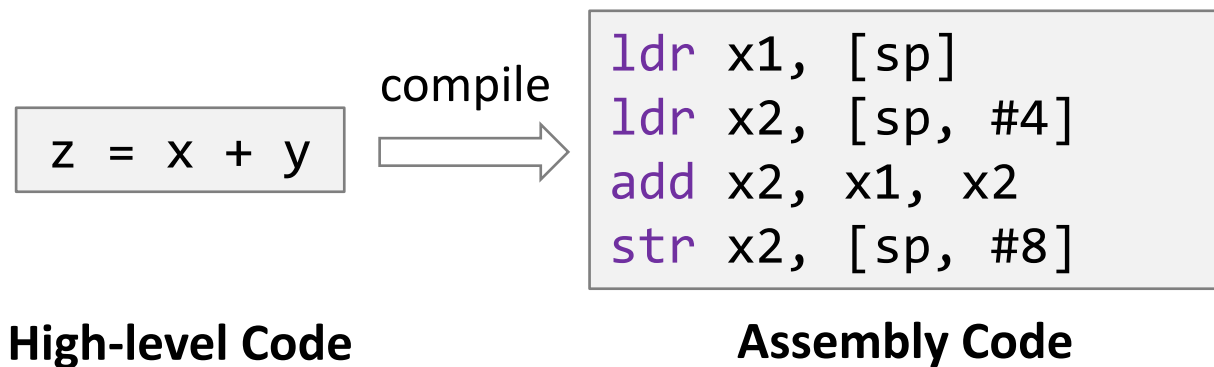
- They are invented for different application scenarios.

❑ Assembly code lacks high-level constructs.

- There are different assembly code for different ISAs
 - Poor portability: code tied to a specific architecture/CPU.
- Writing assembly code is painful.
 - Error-prone: hard to maintain correctness at the instruction level.
 - Low efficiency: far more lines of code, leading to slower development.

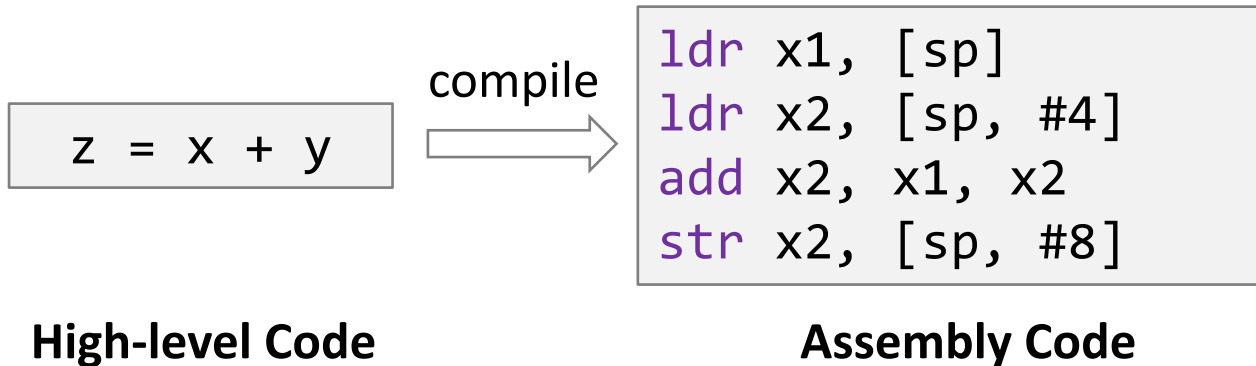
Comparison between High-level and Assembly Code

- ❑ In computer programs, variables are placed on memory.
- ❑ Assembly code has no variable names, but only memory addresses.
- ❑ Access variables via a stack pointer (sp) + offset of the variable.



Features of Compilers

- ❑ Understanding: Parse and interpret the source code.
- ❑ Translation: Generate equivalent assembly code.
- ❑ Optimization: Improve the speed and size of the code.



Challenge: Ambiguity of High-level Languages



Challenge: Ambiguity of High-level Languages



Ambiguity of Code: Example

❑ Code written in programming languages can be ambiguous.

❑ In the following example, what is the result when $a = 1, b = 0$?

```
result = 0
if (a > 0)
    if (b > 0)
        result = 1;
    else
        result = -1;
```

C/C++ Code

```
result = 0
if (a > 0) {
    if (b > 0) {
        result = 1;
    } else {
        result = -1;
    }
}
```

Interpretation 1

```
result = 0
if (a > 0) {
    if (b > 0) {
        result = 1;
    }
} else {
    result = -1;
}
```

Interpretation 2

Programming with Natural Language?

- ❑ Before LLM (Large language model), interpretation is a big problem.
- ❑ LLM makes natural-language programming increasingly promising.

Prompt: Write a python program to calculate the value of y based on the input x.
The rule is:
- if x equals 1 or 2, y equals x;
- otherwise, y equals x times the result of y for x - 2.

Code generated by LLM:

```
def foo(x):  
    if x == 1 or x == 2:  
        return x  
    else:  
        return x * foo(x - 2)
```

Programming with Natural Language

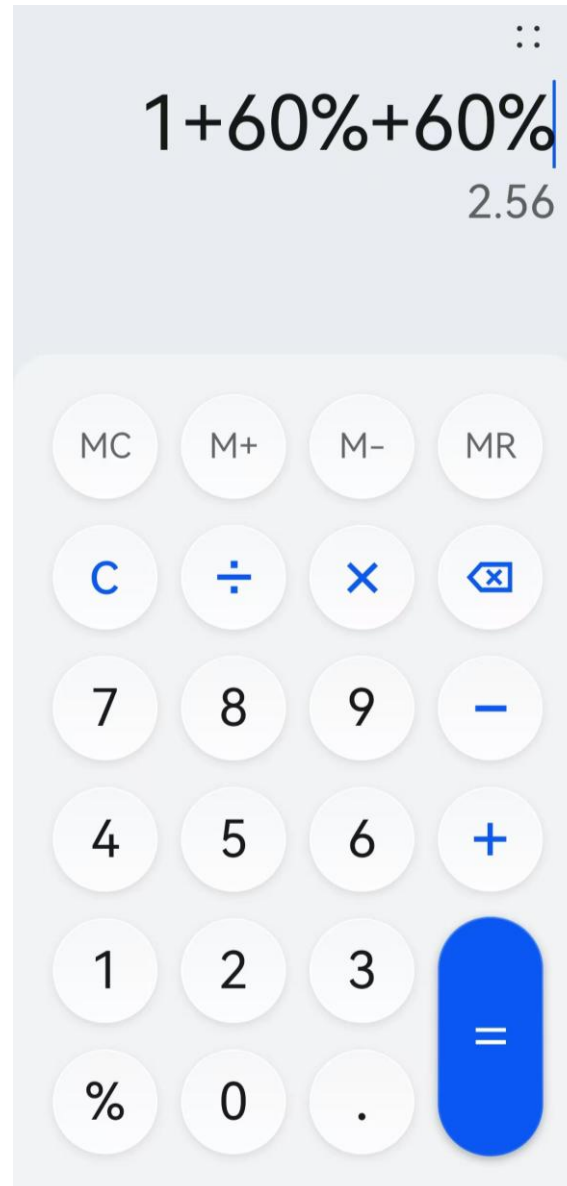
❑ Challenge: Natural language is often vague or underspecified.

- Vague or Ambiguous: The same phrase can mean different things.
- Underspecified: The description isn't self-contained; key details are missing.
 - Example 1: "Write a Fibonacci function." What is the definition of such a function?
 - Example 2: "Write a Crawler that can XXX." What content? What data format?

❑ Pseudocode is more accurate than natural language.

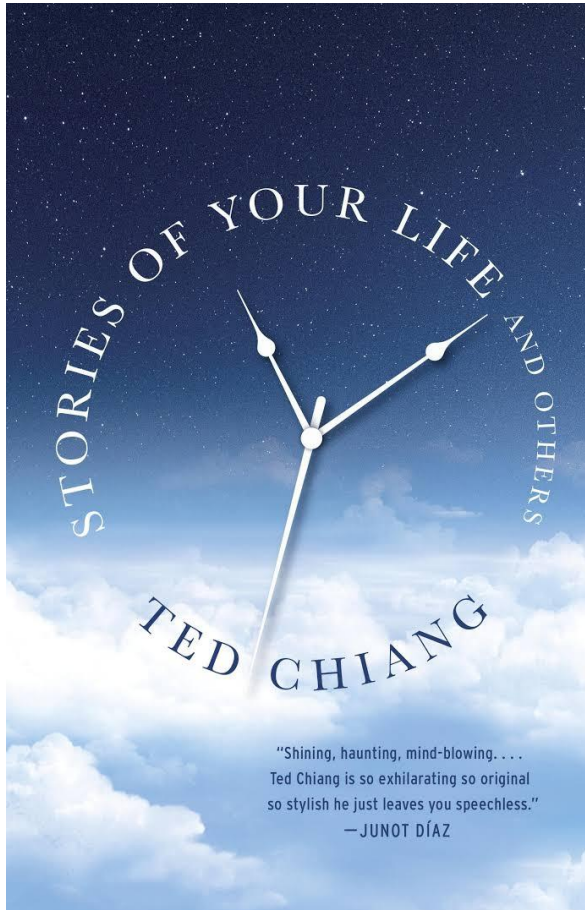
- Describe a program using structured, human-readable statements.
- Do not follow the syntax of any specific programming language.

Underspecified Expressions



Sapir-Whorf Hypothesis: Language Influences Thought

- ❑ The structure and vocabulary of the language you speak can affect how you perceive, categorize, and reason about the world.



The image shows the IMDb page for the movie 'Arrival'. The page has a dark background. At the top, the movie title 'Arrival' is displayed in a large, white, serif font. Below the title, the year '2016', rating 'PG-13', and runtime '1h 56m' are shown. To the right of the title, there are links for 'Cast & crew', 'User reviews', 'Trivia', 'FAQ', 'IMDbPro', and 'All topics'. Below these links, the IMDb rating is shown as a yellow star with '7.9/10' and '882K' votes. To the right of the rating, there is a 'Rate' button and a 'Popularity' section showing '322' and '11'. The main image is a movie poster for 'Arrival' featuring Amy Adams, Jeremy Renner, and Forest Whitaker. Below the poster, there is a video player showing a trailer with a play button and the text 'Play trailer 1:01'. To the right of the video player, there are buttons for '99+ VIDEOS' and '99+ PHOTOS'. Below the video player, there are genre tags: 'Alien Invasion', 'Epic', 'Psychological Drama', 'Sci-Fi Epic', 'Drama', 'Mystery', and 'Sci-Fi'. A synopsis is provided: 'Linguist Louise Banks leads a team of investigators when gigantic spaceships touch down around the world. As nations teeter on the verge of global war, Banks and her crew must find a way to communicate with the extraterrestrial visitors.' Below the synopsis, the director 'Denis Villeneuve' is listed. The writers 'Eric Heisserer' and 'Ted Chiang' are listed. The stars 'Amy Adams', 'Jeremy Renner', and 'Forest Whitaker' are listed. At the bottom, there is a link to 'See production info at IMDbPro'. On the right side of the page, there is a yellow 'Add to Watchlist' button with 'Added by 1.8M users'. Below this, there is a loading spinner. At the bottom right, the user reviews '2.6K' and critic reviews '767' are shown, along with a Metascore of '81'.





Do We Need a New Programming Language?

❑ Reinventing a language can be useful for new scenarios.

- Java: designed for portable execution across platforms.
- Python: excels at data analysis and machine learning.
- Rust: focuses on system software security and safe concurrency.
- XXX: for LLM?



Consider this image before saying "don't reinventing the wheel?"



2. Python Basics

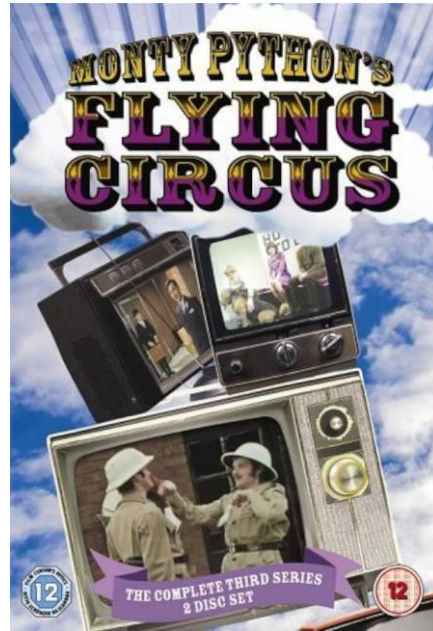
Python



- ❑ Invented by Dutch programmer Guido van Rossum in 1991.
- ❑ Designed as a programming language between C and Shell.
- ❑ Famous for its clean and simple syntax.



Guido van Rossum



Origin of the name
(a British comedy)



Python: Bootloader of LLM

Learning Objective

❑ Before the era of LLM

- Vocabulary: Reserved keywords (e.g., def, if, else), commonly used APIs.
- Syntax: How to write code correctly (e.g., indentation, brackets).
- Grammar: How different language constructs are structured and combined.

❑ You don't need to remember them because of LLM is good at this.

- You don't need to memorize everything—LLMs are good at this.
- But you should still be able to read code.
- So that you can understand and evaluate what LLMs generate.

Variables

- ❑ **A variable is like a label you stick on a box so you can store something and find it later.**
- ❑ **Rules of variable naming**
 - Variable names must start with a letter or underscore, letters, numbers.
 - Avoid Python keywords like 'def', 'for', 'if'.
 - Example of valid names: abc123, _abc, ifabc.
 - Example of invalid names: 123abc.
- ❑ **Case-sensitive: *e.g.*, 'abc123' and 'Abc123' are different.**

Naming Conventions: Snake Case vs Camel Case

❑ How do we name a variable with multiple words?

- Whitespace is not allowed.

❑ Snake case: Words are lowercase and separated with underscores.

- For example: `fudan_university`, `total_score`

- Used in python: variable names, function names

❑ Camel Case: Each word starts with a capital letter.

- For example: `FudanUniversity`, `TotalScore`

- Used in python: class names.

Types of Variables

- ❑ Each variable is typed, such as integer, float, string, *etc.*
- ❑ By default, the type is implicit and automatically inferred.
 - Since Python 3.5, you can annotate your functions and variables with types.

```
age = 25          # integer
height = 1.75     # float
name = "Alice"    # string
age = "25"        # string
is_student = True # boolean
```

Implicit typed variables

```
age: int = 25
height: float = 1.75
name: str = "Alice"
age: str = "25"
is_student: bool = True
```

Explicit typed variables
(since Python 3.5)

Type Checking

❑ Incompatible type: Python Raises TypeError.

- A function requires an integer but receives a string.
- The operation does not make sense for the given types.

❑ Type coercion (implicit conversion):

- Python automatically converts a narrower numeric type to a wider type.
- This allows mixed-type numeric operations.

```
x = 5 + 3           # integer 8
x = "5" + "3"       # string "53"
x = 5 + "3"         # TypeError
x = 0.5 + 3          # Type coercion: float 3.5
x = 5 + True         # Type coercion: integer 6
```

Define a Variable

❑ Variables must be defined (assigned) before used.

Define a variable :

```
score = 10  
score = a           # illegal if a is undefined
```

Update the value of a variable

```
score = score + 5    # score must be defined before  
score += 5           # score = score + 5
```

Define multiple variables in one statement:

```
x = y = z = 0        # x=0; y=0; z=0  
a, b, c = 1, 2, 3    # a=1; b=2; c=3;
```

Conditional Statement: if

- ❑ **Condition:** A logical expression or Boolean expression
- ❑ **Code block:** Executed if the condition evaluates to true.

'if' directive

```
if n == 0:
    do_sth()
    do_others()
```

condition

true branch

Four spaces (or a tab) indicate the scope of the true branch



Conditional Statement: if-else

❑ An if-else statement consists of:

- An if condition and a code block to be executed if the condition is true.
- An else block to be executed if the condition is false.

```
if n == 0:  
    do_sth()  
else:  
    do_else()
```

Question: Is the Following Code Valid?

```
# program start
if random() % 2 == 0:
    b = 1;
else:
    b = 0;
b = b + 1;
```

- ❑ Potential issue: the value of `b` may be used before it is defined.
- ❑ In python: valid because `b` is defined before use in both paths.
- ❑ But it could be invalid in other languages like C/C++ and Rust.

Question: Is the Following Code Valid?

```
# program start
if random() % 2 == 0:
    b = 1;
b = b + 1;
```

☐ **Python uses dynamic checking: errors are detected at runtime.**

- Valid, b is defined if the condition is true.
- Invalid, b is undefined if the condition is false.

Multiple Conditions: if-elif-else Statement

❑ An if-elif-else statement consists of:

- An if condition and the block of code to be executed if the condition is true.
- One or several elif condition(s) to check and corresponding code blocks.
- An else block (optional) to be executed if all previous conditions are false.

❑ Use match-case when pattern matching makes the code clearer.

```
if n == 0:  
    do_sth()  
elif n == 1:  
    do_elif()  
else:  
    do_else()
```

Multiple Conditions: match-case (since Python 3.10)

- ❑ It matches a value or multiple values against different patterns.
- ❑ Makes code more readable for multiple discrete cases.

```
match a:  
    case 0:  
        do_0()  
    case 1:  
        do_1()  
    case _:  
        do_else()
```

```
match (a, b):  
    case (0, 0):  
        do_00()  
    case (1, _):  
        do_1()  
    case (_, _):  
        do_else()
```

Iterations: while Statement

- ❑ Repeat a block of code as long as a given condition is true.
- ❑ It is often used when the number of iterations is not known.

'while' directive

```
result = 1
i = 2
while i <= n:
    result *= i
    i += 1
return result
```

→ loop condition

→ body

Iterations: for Statement

- ❑ Repeat a block of code for each item in a sequence.
- ❑ You don't manually manage loop counters.
- ❑ Python iterates automatically.

'for' directive

```
result = 1
for i in range(2, n + 1):
    result *= i
return result
```

→ loop condition

→ body

Function

- ❑ **An abstraction that maps a set of input values to a set of output.**
- ❑ **A named block of code that performs a specific task.**
 - Take inputs (parameters).
 - Optionally produce an output (return value).
- ❑ **Function acts as a reusable unit of work.**
 - Define once, call multiple times.

Function: Syntax

- ❑ A function generally has a name and a code body.
- ❑ Arguments and return values are optional.

define a function: start with the 'def' keyword

name of the function

name of the argument

```
def fac(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * fac(n - 1)
```

call the function 'fac'

Iteration vs Recursion

❑ **Iteration:** Repeating a block of code using loops.

➤ Doing something repeatedly until a condition is met.

❑ **Recursion:** A function calls itself to solve a problem.

➤ Solving a problem by reducing it to a smaller version of itself.

```
def fac(n):  
    result = 1  
    for i in range(2, n + 1):  
        result *= i  
    return result
```

```
def fac(n):  
    if n == 1:  
        return 1  
    return n * fac(n - 1)
```

More Types in Python

❑ **Scalar type:** A scalar type stores a single value.

➤ Boolean, integer, float, string (immutable in Python)

❑ **Compound type:**

➤ Tuple: An ordered, immutable collection of items

➤ List: A built-in mutable sequence to store an ordered collection of items

➤ Class

List

- ❑ A collection of elements.
- ❑ Similar to an array, but can hold elements of different types.
- ❑ Elements are accessed by index.

```
a = []                # Empty list
b = [1, 2, 3]         # List with elements
c = [1, "hello", 3.14] # List with mixed types
print(b[0])           # access the first element of b
b.append(4)           # Add at the end
b.insert(2, 100)       # Insert 100 at index 1
b.remove(2)           # Remove first occurrence of 2
for n in nums:        # Simple for loop
    print(n)
```

Tuple

- ❑ An ordered, immutable collection of elements.
- ❑ Similar to a list, but cannot be modified after creation.
- ❑ Can hold elements of different types.
- ❑ Tuples are generally more memory-efficient than lists.

➤ Examine this claim with agent.

```
t1 = (1, 2, 3)           # Tuple with elements
t2 = (1, "hello", 3.14)  # Tuple with mixed types
t3 = (1, (2, 3), [4, 5]) # Nested tuple
print(t1[0])             # 1, access the first element of t1
print(t1[0:2])           # 1,2, access the slice of t1
for item in t1:          # access all elements iteratively
    print(item)
```

Dictionary

- ❑ A built-in data structure that stores data as key-value pairs.
- ❑ Dictionary is very efficient for lookups.
- ❑ You can use a key to quickly retrieve or modify the value.

```
students = {  
    "Alice": 20,  
    "Bob": 21,  
    "Charlie": 19  
}  
print(students["Bob"]) # Output: 21
```

Class

❑ **A programming abstraction that defines a new type of object.**

- Data: Attributes that its instances have.
- Methods: Behaviors that its instances can perform.

❑ **Similar to function that acts as a reusable unit of work:**

- Define it once and create multiple instances.
 - Share the same structure and behavior.
 - But can hold different data.

Define a Class

define a class: start with the 'class' keyword

name of the class in Camel Case

constructor of the class

```
class Ellipse:
    def __init__(self, f1, f2, radius_sum):
        self.f1 = f1
        self.f2 = f2
        self.radius_sum = radius_sum
        dist_foci = math.dist(f1, f2)
        if radius_sum <= dist_foci:
            raise ValueError("Not an ellipse.")
    def contains_point(self, x, y):
        ...
```

Use the Class

Create an Ellipse instance via the constructor: `__init__`

```
e = Ellipse((0, 0), (4, 0), 10)
```

Use class methods via the class instance.

```
print(e.contains_point(3, 0))
```

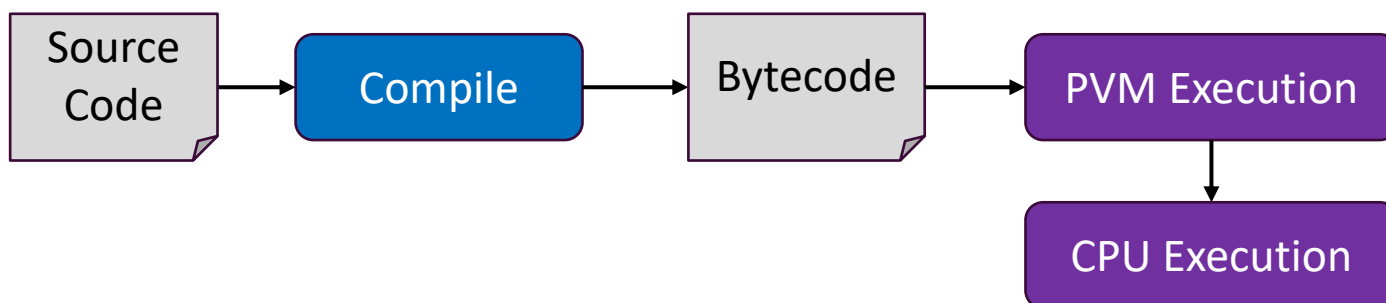
How Python Code is Executed

❑ Source code: .py files

❑ Compilation: Python compiles source code into bytecode.

➤ Bytecode may be cached as .pyc files in the `__pycache__` folder.

❑ Execution: The Python Virtual Machine (PVM) executes the bytecode.



3. Agentic Coding with Python

Vibe Coding or Agentic Coding



Andrej Karpathy ✓

@karpathy · [Follow](#)



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper [Show more](#)

4:47 AM · Feb 3, 2025



28.1K



Reply

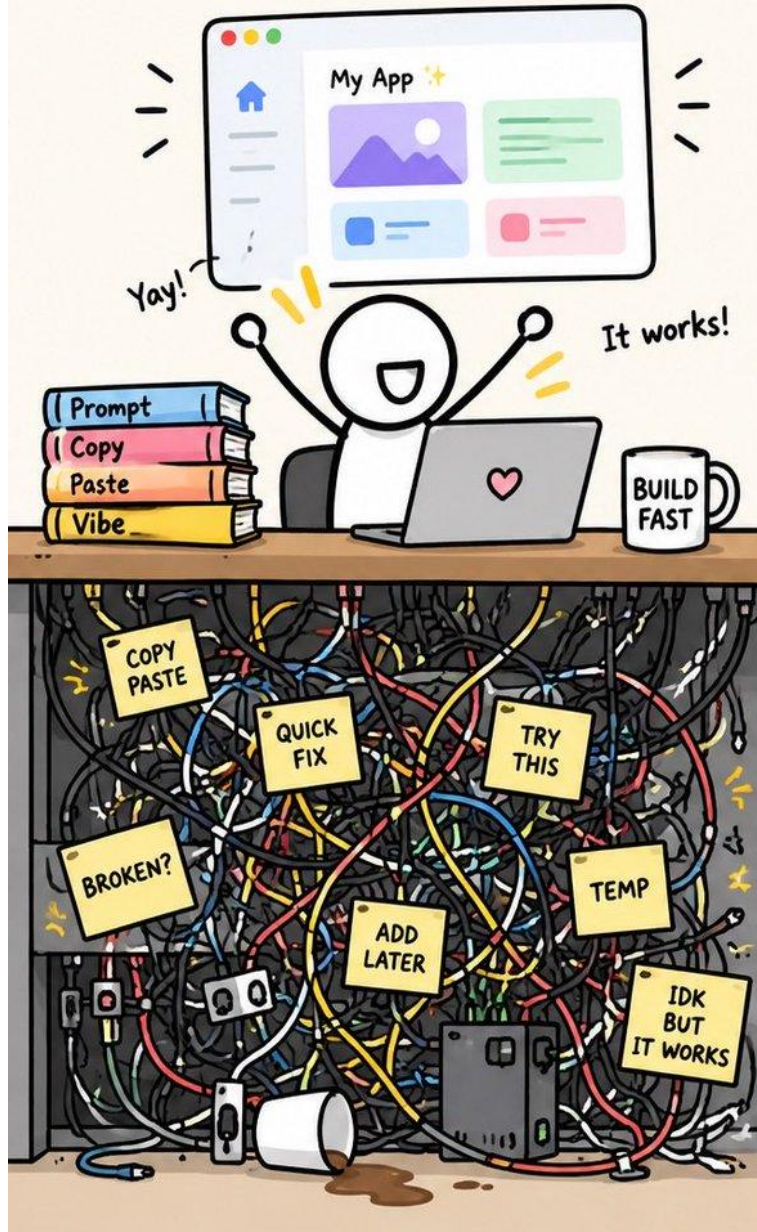


Copy link

[Read 1.3K replies](#)

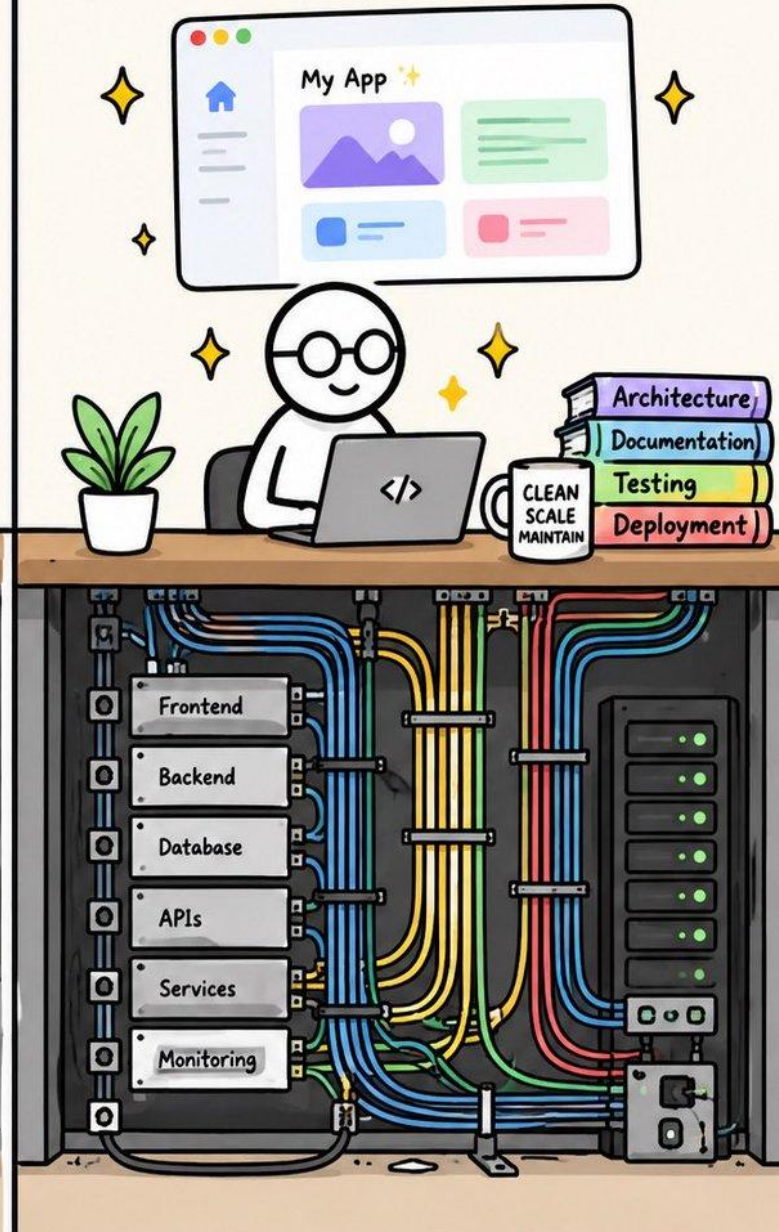
Vibe Coder

Looks good. It works. Ship it!

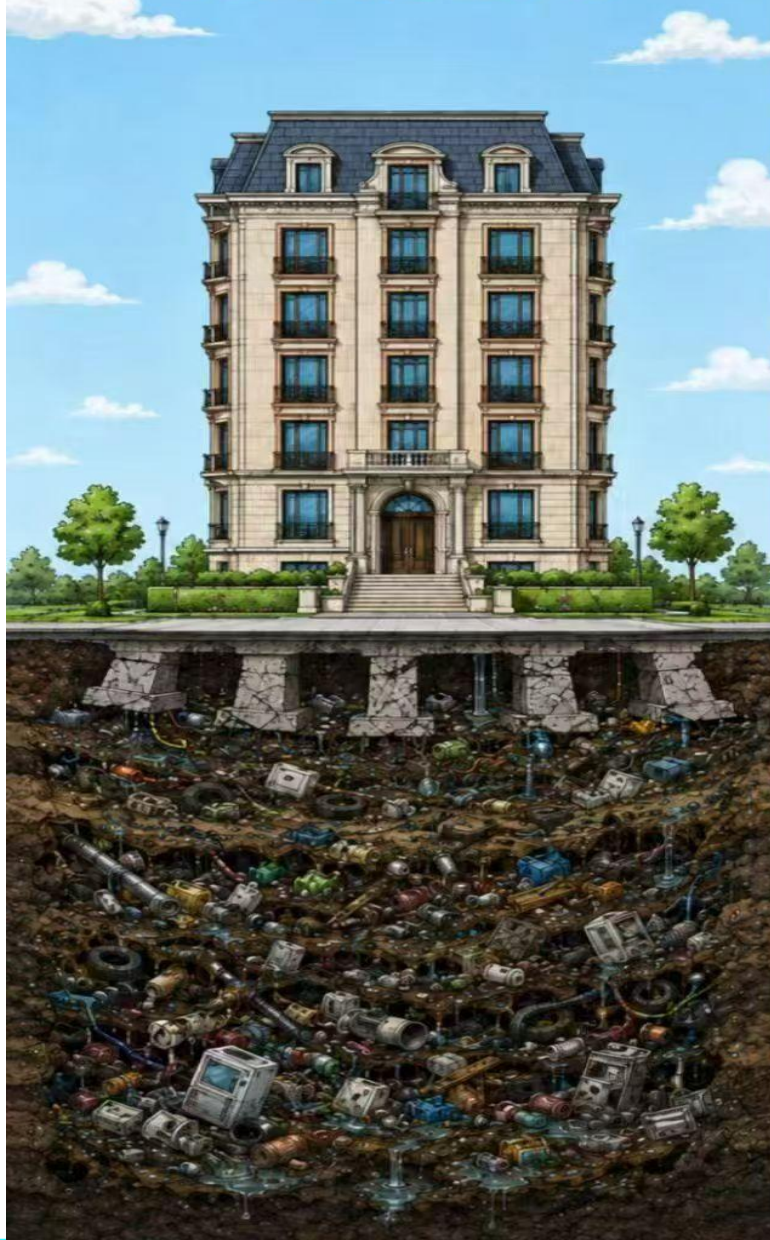


Software Engineer

Plan. Design. Structure. Scale.



VIBE CODING



SOFTWARE ENGINEERING



Install Python with Agent

Installing Python

Install python

Build · DeepSeek V4 Pro · 10:28 PM

Shell

Check Python and winget availability

Shell

Install Python 3.12 via winget

Shell

Verify Python installation

Python 3.12.10 installed. Open a new terminal for **python** to be on your PATH.

Ask anything...

+

↑

Build

DeepSeek V4 Pro

Default

50

A circular blue watermark seal of a university, featuring a central emblem and the year '1905' at the bottom.

Sample Task: Write a Game

Help me build a simple snake game in Python.

Shell Install pygame library

Shell Install pygame with refreshed PATH

Shell Verify pygame installed

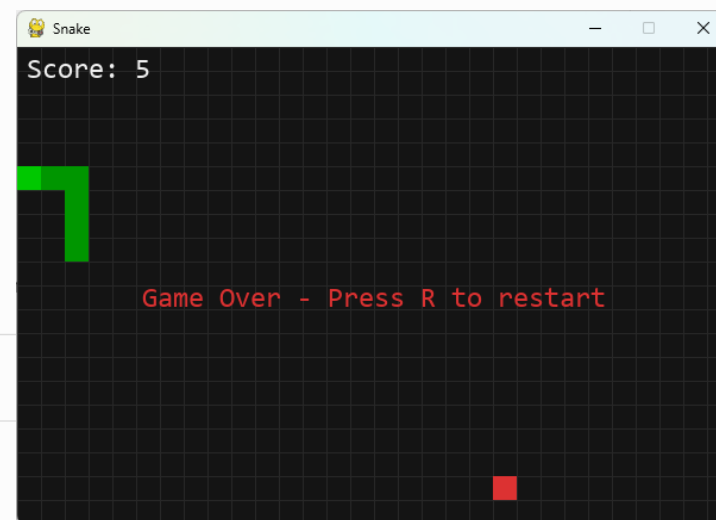
Write snake.py

Shell Syntax-check snake.py

Done. [snake.py](#) in Downloads. Run it with:

```
python snake.py
```

Arrow keys to move, R to restart after game over.



run

Shell Run the snake game

Read the Generated Code

```
import sys
import pygame

CELL = 20
GRID_W = 30
GRID_H = 20
WIDTH = CELL * GRID_W
HEIGHT = CELL * GRID_H
DARK = (40, 40, 40)

def draw_grid(screen):
    for x in range(0, WIDTH, CELL):
        pygame.draw.line(screen, DARK, (x, 0), (x, HEIGHT))
    for y in range(0, HEIGHT, CELL):
        pygame.draw.line(screen, DARK, (0, y), (WIDTH, y))
    ...
```